

Design and Analysis of Algorithms

Lecture 1: Asymptotic Analysis





Table of Contents

- Asymptotic Analysis
- Big-O Notation (O)
- Big-Omega Notation (Ω)
- Big-Theta Notation (Θ)
- Growth-Rate Functions



Excerpts from the previous lecture

- Generally, we use a computer because we need to process a large amount of data. When we run a program on large amounts of input, **besides to make sure the program is correct, we must be certain that the program terminates within a reasonable amount of time.**
- Process of **determining the amount of time**, resources, etc. required when executing an algorithm. **It is only performed on correct algorithms.**



Excerpts from the previous lecture

There are two aspects of **algorithmic performance**:

- **Time complexity:**

- ✓ Instructions take time...
- ✓ How fast does the algorithm perform?
- ✓ What affects its runtime?

- **Space complexity:**

- ✓ Data structures take space...
- ✓ What kind of data structures can be used?
- ✓ How does choice of data structure affect the runtime?

- **We will focus on time:**

- ✓ How to estimate the time required for an algorithm?
- ✓ How to reduce the time required?



Excerpts from the previous lecture

- **The primary goal of this course** is to provide you with a toolbox of techniques for solving complex computational problems efficiently. You will learn how to:
 - ✓ **Analyze an algorithm's resource consumption** (primarily time and memory) in a formal, mathematical way.
 - ✓ **Design new algorithms** by applying well-established patterns and principles.
 - ✓ **Classify computational problems** based on their inherent difficulty and understand the limits of what computers can efficiently solve.

This course moves you from being a programmer who implements solutions to a computer scientist who creates and evaluates them.



How to Compare Algorithms? (1/3)

- **Naïve Approach - Experimental Approach**

- Code it, run it with different inputs, and measure the time.
- Problems:
 - Hardware Dependent: A fast computer can make a bad algorithm look good.
 - Implementation Dependent: A cleverly coded $O(n^2)$ might beat a poorly coded $O(n \log n)$.
 - Data Dependent: Performance can vary wildly based on the specific input (e.g., sorted vs. unsorted).



How to Compare Algorithms? (2/3)

- **Theoretical (Asymptotic) Analysis**

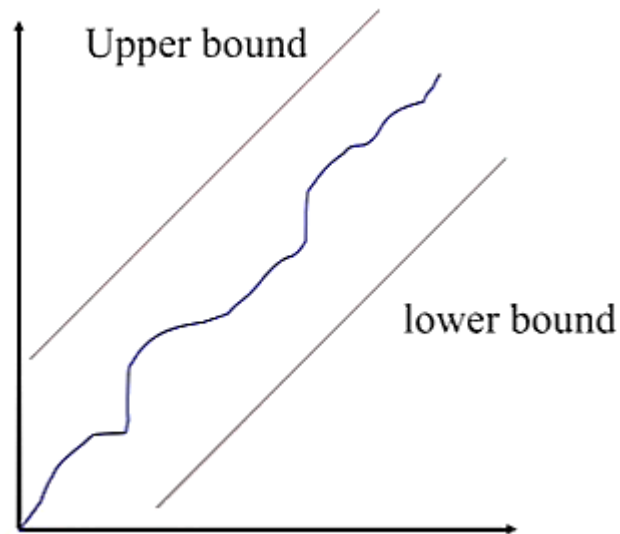
- When we analyze algorithms, we should employ mathematical techniques independent of specific implementations, computers, or data.
- Based on high-level description of the algorithms, rather than language dependent implementations.
- Makes possible an evaluation of the algorithms that is independent of the hardware and software environments



How to Compare Algorithms? (3/3)

- **Theoretical (Asymptotic) Analysis**

- Count the number of primitive operations (a single step like addition, comparison, assignment) as a function of the input size n . This gives us a function, $T(n)$.
- Describe $T(n)$ using **Asymptotic Notation** (worst case (Big-O), best case (Omega), average case (Theta)) to express its **growth rate**.



Algorithm Description

- **Pseudo-Code** is a description of an algorithm that is less formal than a programming language. It uses the structural conventions of programming languages but is intended for human reading, not machine execution.
- **Example**: Write an algorithm in pseudo-code to Find the maximum element in an array.

Algorithm arrayMax(A, n):

Input: An array A storing n integers

Output: The maximum element in A

currentMax $\leftarrow$ A[0]

for i $\leftarrow$ 1 to n-1 do

 if currentMax < A[i] then currentMax $\leftarrow$ A[i]

return currentMax



Pseudo Code- Core Syntax and Conventions (1/2)

- ✓ **Use standard mathematical symbols** (e.g., +, -, *, /).
- ✓ **Assignment:** $\leftarrow$ (e.g., $x \leftarrow 10$)
- ✓ **Equality Comparison:** = (e.g., if $x = 5$ then)
- ✓ **Algorithm Name(param1, param2)** such as Algorithm CalculateAverage(scores, n)



Pseudo Code- Core Syntax and Conventions (2/2)

- ✓ **Decision** (if/else): if ... then ... [else ...]
- ✓ **While Loop**: while ... do
- ✓ **Repeat Loop**: repeat ... until ... (condition is checked after the loop body)
- ✓ **For Loop**: for ... do
- ✓ **Array Indexing**: A[i]
- ✓ **Method Calls**: object.method(arguments)
- ✓ **Return Statement**: return value



Low Level Algorithm Analysis (1/2)

- This approach analyzes algorithms by counting the number of primitive operations they perform. These are low-level computations that are independent of any programming language.
- **Each of the following is typically counted as one primitive operation:**
 - ✓ Performing an arithmetic operation (e.g., addition, subtraction)
 - ✓ Calling a method or returning from a method
 - ✓ Indexing into an array ($A[i]$)
 - ✓ Making a comparison ($x < y$)



Low Level Algorithm Analysis (2/2)

- **The analysis method is performed by:**
 - ✓ Inspecting the pseudo-code.
 - ✓ Counting the total number of these primitive operations the algorithm executes.
- This count provides a concrete, quantitative measure of an algorithm's work, which is directly related to its running time.



Formulating the Execution Time of Algorithms (1/4)

- For a sequence of statements, the total cost is the sum of the costs for each operation.
As the following example.

`count = count + 1;`

Cost: c_1

`Result = Fact * count;`

Cost: c_2

- $Total\ Cost = c_1 + c_2$



Formulating the Execution Time of Algorithms (2/4)

- When analyzing conditional statements, we consider the worst-case path.

<code>if (n < 0)</code>	Cost: c_1 , Times: 1
<code>absval = -n</code>	Cost: c_2 , Times: 1
<code>else</code>	
<code>absval = n;</code>	Cost: c_3 , Times: 1

- $Total\ Cost \leq c_1 + \max(c_2, c_3)$
- This represents the worst-case execution time, where the algorithm takes the slower of the two possible branches.



Formulating the Execution Time of Algorithms (3/4)

<code>i = 1;</code>	Cost: c_1 ,	Times: 1
<code>sum = 0;</code>	Cost: c_2 ,	Times: 1
<code>while (i <= n) {</code>	Cost: c_3 ,	Times: $n+1$
<code>i = i + 1;</code>	Cost: c_4 ,	Times: n
<code>sum = sum + i;</code>	Cost: c_5 ,	Times: n
<code>}</code>		

- $Total\ Cost = c_1 + c_2 + (n + 1) \cdot c_3 + n \cdot c_4 + n \cdot c_5$
- The time required for this algorithm is proportional to n



Formulating the Execution Time of Algorithms (4/4)

<code>i=1;</code>	Cost: c_1 , Times: 1
<code>sum = 0;</code>	Cost: c_2 , Times: 1
<code>while (i <= n) {</code>	Cost: c_3 , Times: $n+1$
<code>j=1;</code>	Cost: c_4 , Times: n
<code>while (j <= n) {</code>	Cost: c_5 , Times: $n \times (n+1)$
<code>sum = sum + i;</code>	Cost: c_6 , Times: $n \times n$
<code>j = j + 1;</code>	Cost: c_7 , Times: $n \times n$
<code>}</code>	
<code>i = i + 1;</code>	Cost: c_8 , Times: n
<code>}</code>	

- $Total\ Cost = c_1 + c_2 + (n + 1) \cdot c_3 + n \cdot c_4 + n \cdot (n + 1) \cdot c_5 + n \cdot n \cdot c_6 + n \cdot n \cdot c_7 + n \cdot c_8$
- The time required for this algorithm is proportional to n^2 .



General Rules for Estimation

- **Loops:** The running time of a loop is at most the running time of the statements inside of that loop times the number of iterations.
- **Nested Loops:** Running time of a nested loop containing a statement in the inner most loop is the running time of statement multiplied by the product of the size of all outer loops.
- **Consecutive Statements:** Just add the running times of those consecutive statements.
- **If/Else:** Never more than the running time of the test plus the larger of running times of S1 and S2.



Algorithm Growth Rates

- We measure an algorithm's time requirement as a function of the problem size. Problem size depends on the application: e.g. number of elements in a list for a sorting algorithm.
- So, for instance, we say that (if the problem size is n)
 - ✓ Algorithm A requires $5 \cdot n^2$ time units to solve a problem of size n .
 - ✓ Algorithm B requires $7 \cdot n$ time units to solve a problem of size n .
- The most important thing to learn is how quickly the algorithm's time requirement grows as a function of the problem size.
 - ✓ Algorithm A requires time proportional to n^2 .
 - ✓ Algorithm B requires time proportional to n .
- An algorithm's formulated proportional time requirement is known as growth rate.
- We can compare the efficiency of two algorithms by comparing their growth rates.



Common Growth Rates

Growth Rate	Notation	Name	Example Algorithms
Constant	$O(1)$	Constant	Array access, arithmetic operations
Logarithmic	$O(\log n)$	Logarithmic	Binary search, balanced tree operations
Linear	$O(n)$	Linear	Finding max/min, linear search
Linearithmic	$O(n \log n)$	Linear $\times$ Logarithmic	Merge sort, heap sort, quick sort (average)
Quadratic	$O(n^2)$	Quadratic	Bubble sort, insertion sort, naive matrix multiplication
Cubic	$O(n^3)$	Cubic	Naive matrix multiplication (3 nested loops)
Polynomial	$O(n^k)$	Polynomial	Algorithms with k nested loops
Exponential	$O(2^n)$	Exponential	Traveling salesman (brute force)
Factorial	$O(n!)$	Factorial	Traveling salesman (naive), permutation generation



What to Analyze? (1/2)

- **Worst-Case Analysis:** The maximum amount of time that an algorithm require to solve a problem of size n .
 - ✓ This gives an upper bound for the time complexity of an algorithm.
 - ✓ Normally, we try to find worst-case behavior of an algorithm.
- **Best-Case Analysis:** The minimum amount of time that an algorithm require to solve a problem of size n .
 - ✓ The best-case behavior of an algorithm is NOT so useful.
- **Average-Case Analysis :** The average amount of time that an algorithm require to solve a problem of size n .
 - ✓ Sometimes, it is difficult to find the average-case behavior of an algorithm.
 - ✓ We have to look at all possible data organizations of a given size n , and their distribution probabilities of these organizations.
- **Worst-case analysis is more common than average-case analysis.**



What to Analyze? (2/2)

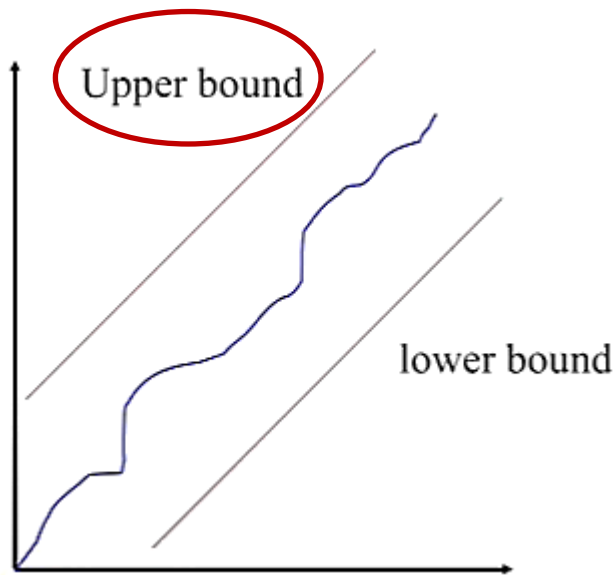
- Let's use substitution with actual values to clarify the three cases for $f(n) = 2n$.

n_0	$f(n) = 2n$
1	2
2	4
3	6
4	8
5	10
1000	2000



Big-O Notation (O)

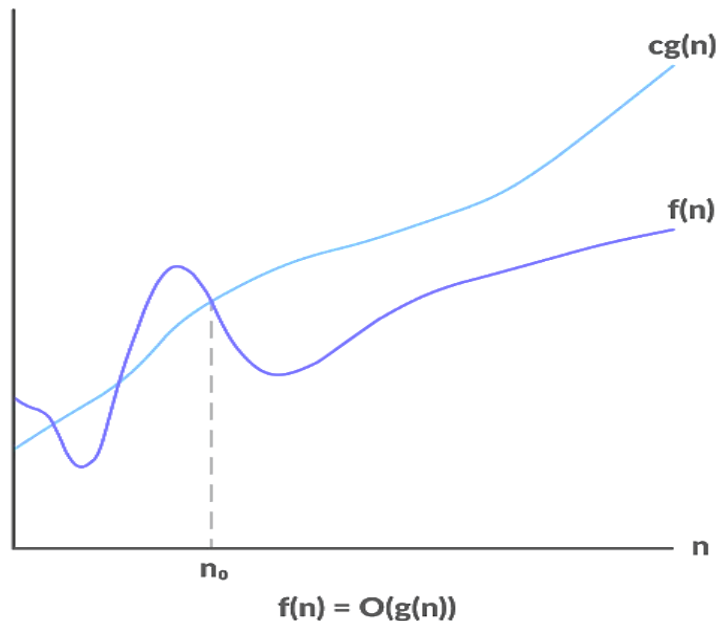
- Known **as worst-case scenario**.
- Also known as **the upper bound**.
- It is a theoretical measure of the execution of an algorithm, usually the time, given the problem size n , which is usually the number of items.
- Notation example: $f(n) = O(g(n))$
 - ✓ Informally, saying some equation $f(n) = O(g(n))$ means it is less than some constant multiple of $g(n)$.



Big-O Notation (O)- Formal Definition

- $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c * g(n)$ for all $n \geq n_0$.
- The growth rate of $f(N)$ is less than or equal to the growth rate of $g(N)$.
- $g(N)$ is an upper bound on $f(N)$.
- **Example:**

$T(n) = 3n^2 + 2n + 5$ is $O(n^2)$. We can choose $c = 4$ and $n_0 = 3$. For all $n > 3$, $3n^2 + 2n + 5 \leq 4n^2$.



Big-O Notation (O)- Examples

- $n^2 + 3n + 4 \rightarrow O(n^2)$, since $n^2 + 3n + 4 < 2n^2$ for all $n > 10$.
- $n^3 + 3n + 4 \rightarrow O(n^3)$
- $(n^2 / 2) - 3n \rightarrow O(n^2)$
- $1 + 4n \rightarrow O(n)$
- $7n^2 + 10n + 3 \rightarrow O(n^2) \rightarrow O(n^3)$
- $\log_{10} n = \log_2 n / \log_2 10 \rightarrow O(\log_2 n) \rightarrow O(\log n)$
- $\sin n \rightarrow O(1)$; $10 \rightarrow O(1)$, $10^{10} \rightarrow O(1)$
- $\log n + n \rightarrow O(n)$

The statement uses the logarithmic identity:
 $\log_a b = \log_x b / \log_x a$
Applying this to our case:
 $\log_{10} n = \log_2 n / \log_2 10$

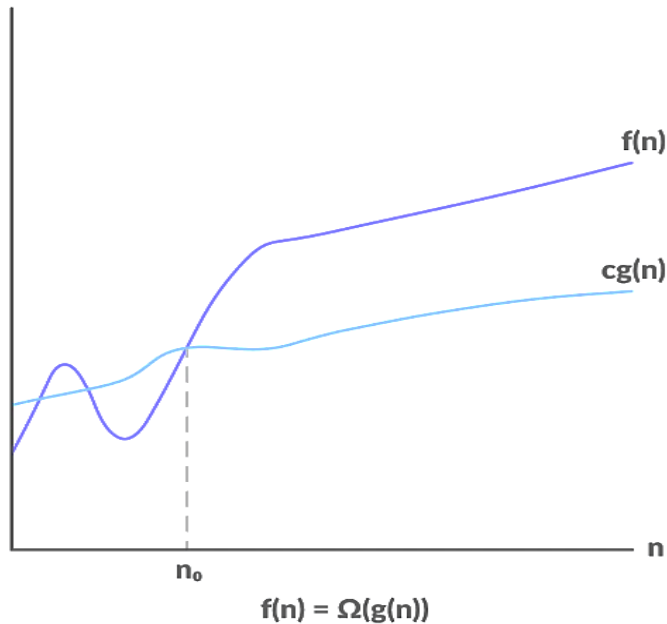
Constant



Big-Omega Notation (Ω)

- Known as **best-case scenario**.
- Also known as **the lower bound**.
- The growth rate of $f(N)$ is greater than or equal to the growth rate of $g(N)$.
- $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq c * g(n) \leq f(n) \text{ for all } n \geq n_0.$$



Big-Omega Notation (Ω)- Example

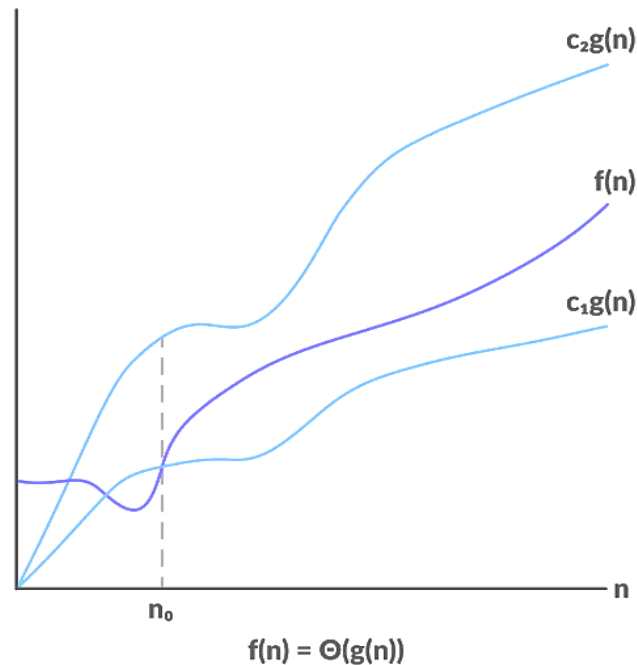
- **Let $f(N) = 2n^2$. Then:**
 - ✓ $f(n) = \Omega(n^2)$ (best answer)
 - ✓ $f(n) = \Omega(n)$



Big-Theta Notation (Θ)

- Known as **average-case scenario**.
- Also known as **the tight bound**.
- $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
- **Example:**

$T(n) = 3n^2 + 2n + 5$ is $\Theta(n^2)$ because it is both $O(n^2)$ and $\Omega(n^2)$.



Big-Theta Notation (Θ)- Example

- Let $f(N) = 2n^2, g(N) = n^2$
 - ✓ Since $f(N) = O(g(N))$ for $c_2 > 2$ and,
✓ $f(N) = \Omega(g(N))$ for $c_1 < 2$, thus
✓ $f(N) = \Theta(g(N))$.
- Big-Theta means the bound is the tightest possible.



General Rules (1/2)

- **Loops**

- ✓ at most the running time of the statements inside the loop (including tests) times the number of iterations.

- **Nested loops**

- ✓ the running time of the statement multiplied by the product of the sizes of all the loops.

- **Examples:**

```
for (i=0;i<n;i++)  
  for (j=0;j<n;j++)  
    k++;
```

$O(N^2)$

```
for (i=0;i<n;i++)  
  a[i]=0;  
for (i=0;i<n;i++)  
  for (j=0;j<n;j++)  
    a[j] += a[j]+i+j;
```

$O(N)+O(N^2)= O(N^2)$



General Rules (2/2)

- **Consecutive statements**

- ✓ These are just addition.

- **If Statement**

- ✓ If S1 Else S2

- larger of the running times of S1 and S2.



Properties of Growth-Rate Functions

- **We can ignore low-order terms in an algorithm's growth-rate function.**
 - ✓ If an algorithm is $O(n^3 + 4n^2 + 3n)$, it is also $O(n^3)$.
 - ✓ We only use the higher-order term as algorithm's growth-rate function.
- **We can ignore a multiplicative constant in the higher-order term of an algorithm's growth-rate function.**
 - ✓ If an algorithm is $O(5n^3)$, it is also $O(n^3)$.
- **$O(f(n)) + O(g(n)) = O(f(n) + g(n))$**
 - ✓ We can combine growth-rate functions.
 - ✓ If an algorithm is $O(n^3) + O(4n)$, it is also $O(n^3 + 4n) \rightarrow$ So, it is $O(n^3)$.
 - ✓ Similar rules hold for multiplication.



Growth-Rate Functions – Example (1/3)

<code>i = 1;</code>	Cost: c_1 ,	Times: 1
<code>sum = 0;</code>	Cost: c_2 ,	Times: 1
<code>while (i <= n) {</code>	Cost: c_3 ,	Times: $n+1$
<code>i = i + 1;</code>	Cost: c_4 ,	Times: n
<code>sum = sum + i;</code>	Cost: c_5 ,	Times: n
<code>}</code>		

$$\begin{aligned}T(n) &= c_1 + c_2 + (n+1)*c_3 + n*c_4 + n*c_5 \\&= (c_3 + c_4 + c_5)*n + (c_1 + c_2 + c_3) \\&= a*n + b\end{aligned}$$

- So, the growth-rate function for this algorithm is $O(n)$.



Growth-Rate Functions – Example (2/3)

<code>i = 1;</code>	Cost: c1, Times: 1
<code>sum = 0;</code>	Cost: c2, Times: 1
<code>while (i <= n) {</code>	Cost: c3, Times: n+1
<code>j = 1;</code>	Cost: c4, Times: n
<code>while (j <= n) {</code>	Cost: c5, Times: n*(n+1)
<code>sum = sum + i;</code>	Cost: c6, Times: n*n
<code>j = j + 1;</code>	Cost: c7, Times: n*n
<code>}</code>	
<code>i = i + 1;</code>	Cost: c8, Times: n
<code>}</code>	

$$\begin{aligned}T(n) &= c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5 + n*n*c6 + n*n*c7 + n*c8 \\&= (c5 + c6 + c7)*n^2 + (c3 + c4 + c5 + c8)*n + (c1 + c2 + c3) \\&= a*n^2 + b*n + c\end{aligned}$$

- So, the growth-rate function for this algorithm is $O(n^2)$.



Growth-Rate Functions – Example (3/3)

<code>i = 2;</code>	Cost: c_1 , Times: 1
<code>while (i $\leq$ n^2) {</code>	Cost: c_2 , Times: n^2
<code>print "hello";</code>	Cost: c_3 , Times: n^2
<code>print (i);</code>	Cost: c_4 , Times: n^2
<code>i++;</code>	Cost: c_5 , Times: n^2
<code>}</code>	

$$\begin{aligned}T(n) &= c_1 + n^2 * c_2 + n^2 * c_3 + n^2 * c_4 + n^2 * c_5 \\&= c_1 + n^2 * (c_2 + c_3 + c_4 + c_5) \\&= a * n^2 + b\end{aligned}$$

- So, the growth-rate function for this algorithm is $O(n^2)$.



Calculating $T(n)$ Using Summation Equations (1/7)

- **Fundamental Summation Rules for Algorithm Analysis**

Rule 1: Sum of Constant 1

$$\sum_{i=1}^n 1 = n$$

$$T(n) = O(n)$$

Rule 2: Sum of Constant C

$$\sum_{i=1}^n C = C \cdot n$$

$$T(n) = O(n)$$



Calculating $T(n)$ Using Summation Equations (2/7)

- **Fundamental Summation Rules for Algorithm Analysis**

Rule 3: Sum of Index Variable (Arithmetic Series)

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$T(n) = O(n^2)$$

Rule 4: General Polynomial Sum Rule

$$\sum_{i=1}^n i^k = O(n^{k+1}) \quad \text{for } k \geq 0$$



Calculating T(n) Using Summation Equations (3/7)

- **Fundamental Summation Rules for Algorithm Analysis**

Rule 5: Logarithmic Number of Terms

$$\sum_{i=1}^{\log_2 n} C = C \cdot \log_2 n$$

$$T(n) = O(\log n)$$

Rule 6: Harmonic Series

$$\sum_{i=1}^n \frac{1}{i} = O(\log n)$$



Calculating $T(n)$ Using Summation Equations (4/7)

- **Example (1)**

for(i = 1; i <= n; i++)
 print i;

→ $T(n) = \sum_{i=1}^n 1 = n = O(n)$

- **Example (2)**

for(int i = n - 1; i >= 1; i--)
 print i;

→ $T(n) = \sum_{i=1}^{n-1} 1 = n - 1 = O(n)$



Calculating $T(n)$ Using Summation Equations (5/7)

- **Example (3)**

for($i = 1$; $i \leq n$; $i *= 2$)
 print i ;

$$\rightarrow T(n) = \sum_{i=1}^{\log_2 n} 1 = \log_2 n = O(\log n)$$

- **Example (4)**

for($i = n$; $i >= 1$; $i /= 2$)
 print i ;

$$\rightarrow T(n) = \sum_{i=1}^{\log_2 n} 1 = \log_2 n = O(\log n)$$



Calculating T(n) Using Summation Equations (6/7)

- **Example (5)**

```
for(i = 1; i <= n; i++)
```

```
    for(j = 1; j <= n; j++)
```

```
        print "hello";
```

$$\rightarrow T(n) = \sum_{i=1}^n \sum_{j=1}^n 1 = \sum_{i=1}^n n = n \cdot n = n^2 = O(n^2)$$

- **Example (6)**

```
for( i = 1; i <= n; i++)
```

```
    for(j = 1; j <= i; j++)
```

```
        print "hello";
```

$$\rightarrow T(n) = \sum_{i=1}^n \sum_{j=1}^i 1 = \sum_{i=1}^n i = \frac{n(n+1)}{2} = O(n^2)$$



Calculating $T(n)$ Using Summation Equations (7/7)

- Example (7)

```
for( i = 1; i <= n; i *= 3)
    for( j = 1; j <= n; j++)
        print "hello";
```

$$\rightarrow T(n) = \sum_{i=1}^{\log_3 n} \sum_{j=1}^n 1 = \sum_{i=1}^{\log_3 n} n = n \cdot \log_3 n = O(n \log_3 n)$$

- Example (8)

```
for( i = 1; i <= n; i *= 3)
    for( j = 1; j <= n; j *= 3)
        print "hello";
```

$$\rightarrow T(n) = \sum_{i=1}^{\log_3 n} \sum_{j=1}^{\log_3 n} 1 = \log_3 n \cdot \log_3 n = O(\log_3^2 n)$$



Test Your Knowledge (1/2)

1. Order the following functions by growth rate:

$$N, N^N, N^2, N \log N, N!, 2^N, 37, N^3$$

Correct ordering by growth rate:

$$37 < N < N \log N < N^2 < N^3 < 2^N < N! < N^N$$

2. Order the following functions by growth rate:

$$2^N, N^{3/2}, N \log N, N^{\log N}$$

Correct ordering by growth rate:

$$N \log N < N^{3/2} < N^{\log N} < 2^N$$



Test Your Knowledge (2/2)

3. For each of the following program fragments give an analysis of the running time using sigma notation and then write it as Big-Oh

a) `for(i = 0; i < n; ++i)` $\rightarrow$ $T(n) = \sum_{i=0}^{\log_k n} 1 = \log_k n = O(\log_k n)$
 `i *= k;`

b) `sum = 0;`
 `for(i = 0; i < n; ++i)`
 `for(j = 0; j < i * i; ++j)`
 `for(k = 0; k < j; ++k)`
 `++sum;`

$\rightarrow T(n) = \sum_{i=0}^n \sum_{j=0}^{i^2} \sum_{k=0}^j 1 = \sum_{i=0}^n \sum_{j=0}^{i^2} j = \sum_{i=0}^n \frac{i^2(i^2 + 1)}{2} = O(n^5)$



Any Question?

